
Active Dimension as a Lightweight Measure of the Complexity of Neural Network Training Dynamics from Scalar Training Logs

Anonymous Author
Anonymous Institution

Abstract

Neural network training can restrict parameter motion, reduce information stored in a representation, or produce more regular behavior. Tracking these changes through full network states can be expensive. We propose a method for detecting such simplification from a single scalar time series. The method combines delay reconstruction, a pooled nearest-neighbor dimension estimator, and a calibrated change detector. Delay-embedding theory motivates its interpretation in terms of active components. Controlled experiments test this interpretation: in trained recurrent generators, one neuron’s activity distinguishes independent phases from harmonics that spectral peak counts cannot separate. In applied tasks, we validate changes in the estimate against independent measurements. A detector calibrated on a small CNN identifies 27 of 28 strong training interventions with no alarms in 16 control records; the same threshold detects backbone freezing in ResNet-18. In a text VAE, the estimate tracks both latent information loss and its partial preservation across nine seeds. Analysis takes 9.5 ms per CNN log window and is 14–19 times faster than a full Lyapunov spectrum in a recurrent-network benchmark. These results establish a low-cost diagnostic of several forms of learned simplification, with a component interpretation in suitable recurrent regimes.

1 INTRODUCTION

A scalar training log records more than whether a loss or parameter norm rises or falls. Its fluctuations can reflect several interacting degrees of freedom. As learning progresses, some of these may disappear or become coordinated. Detecting that change from the log would help monitor training without storing every parameter vector or repeatedly analyzing the full network state.

We develop a method for detecting such changes from one scalar time series. The starting point is delay-embedding theory: under suitable assumptions, a vector of delayed observations preserves the geometry of the underlying dynamics (Takens, 1981; Sauer et al., 1991; Stark, 1999; Stark et al., 2003). A dimension estimator applied to these vectors can then measure how many independent local coordinates the dynamics require. We use the pooled MacKay–Ghahramani (MG) form (MacKay and Ghahramani, 2005) of the Levina–Bickel estimator (Levina and Bickel, 2004).

Our hypothesis is that this geometric information remains useful when the exact dimension interpretation is unavailable. Restricting parameter updates, suppressing a latent information pathway, or learning a more regular behavior may each reduce the complexity of a scalar record. We test these forms of simplification against independent measurements. For stochastic or transient training, the question is whether changes in MG track the specified outcome reliably. An exact component count is not required for this monitoring task.

Contributions.

1. **A method for monitoring simplification from scalar logs.** We combine MG with a specified observation and windowing procedure, numerical checks, and a change detector that uses only past data. Once the log is available, analysis depends on the window and embedding size, not the model’s parameter count.

2. **Controlled tests of the active-component interpretation.** We test recovery in analytic oscillators and learning problems, including an MLP-based task. In trained recurrent generators, MG from one neuron separates independent phases from harmonics even when spectral peak counts and covariance ranks are similar.
3. **Independent validation in applied learning tasks.** MG detects 27/28 strong CNN interventions with no control alarms, transfers to backbone freezing in ResNet-18, and tracks both information loss and preservation in a text VAE. We compare it with simple scalar statistics using the same records and calibration procedure.
4. **Measured savings in analysis and storage.** We separate the cost of obtaining a log from processing it. A CNN window takes 9.5ms to analyze; in a learned recurrent system, MG is 14–19 times faster than the full Lyapunov spectrum while detecting the same transition.

The methodological contribution is the combination of scalar observation, calibrated estimation, and change detection for learning dynamics. Delay embeddings and local dimension estimators are established tools. We show how to use them for this task, test their interpretation, and measure their benefit against independent references and competing diagnostics.

Relation to existing measurements. Intrinsic-dimension methods usually analyze clouds of data points or representations (Facco et al., 2017; Ansuini et al., 2019; Pope et al., 2021). Subspace-training methods study how many parameter directions suffice to learn a task (Li et al., 2018; Aghajanyan et al., 2021). We instead analyze the temporal structure of a scalar record. Covariance participation ratio measures how variance is distributed across directions (Gao et al., 2017), while Hessian spectra describe local curvature (Gur-Ari et al., 2018; Cohen et al., 2021). Both provide useful independent comparisons. Since simple scalar statistics also avoid full-state storage, we compare MG with roughness, autocorrelation, spectral entropy, and surrogate records (Schreiber and Schmitz, 1996) to test what its geometric analysis adds.

2 ESTIMATING COMPLEXITY FROM A SCALAR TIME SERIES

2.1 Problem statement

Given one scalar observation per step, we want to detect when the underlying dynamics become simpler. Let s_t be the system state and $x_t = \phi(s_t)$ the observation. During optimization, s_t includes the parameters

and optimizer variables. When evaluating a trained recurrent generator, the weights are fixed and s_t is the evolving network activity. These are two different trajectories to which we apply the same estimator.

We consider two tasks. In a recurrent regime with a well-defined dimension, we estimate the number of active components from a finite window of x_t . In nonstationary training, we test whether changes in the estimate track simplification measured independently of MG.

For the first task, let μ be an ergodic occupation measure: $\mu(B(s, \epsilon))$ describes the fraction of time spent within distance ϵ of state s . We define active dimension by

$$d_{\text{act}} = \lim_{\epsilon \downarrow 0} \frac{\log \mu(B(s, \epsilon))}{\log \epsilon}, \quad (1)$$

when the limit exists and has the same value almost everywhere. On a smooth d -dimensional manifold with positive regular density, $d_{\text{act}} = d$. An oscillator with r independent phases has r active components even if its output contains many harmonics. Thus a component is an independent coordinate of the dynamics, not simply a peak in the signal’s spectrum.

For the second task, we hold the observable and estimator settings fixed across windows. Each experiment defines its reference before examining MG: restricted parameter updates, reduced latent information, or more regular learned behavior. These are measurable forms of simplification, but they need not share a single dimension or improve predictive performance. For example, losing useful latent information is simplification that a monitoring method should detect even though it is undesirable.

2.2 The geometric information in delayed observations

A single observation can mix several components. Delayed observations can separate them by recording how the signal changes over time. We form the vector

$$y_t = (x_t, x_{t+\tau}, \dots, x_{t+(E-1)\tau}) \quad (2)$$

with E coordinates separated by lag τ . The Takens and Sauer results give conditions under which generic smooth observations distinguish the states of a deterministic system (Takens, 1981; Sauer et al., 1991). Stark’s extensions address forced systems under additional assumptions (Stark, 1999; Stark et al., 2003). These results explain why a scalar record can retain information about the underlying dynamics. They do not prescribe a finite-sample window or guarantee reconstruction for every SGD trajectory.

The condition needed for our dimension interpretation

is precise. If the delay map is bi-Lipschitz on the sampled support, it changes distances by at most fixed factors. Those factors disappear in the limit in Eq. (1), so the local dimension is preserved. Appendix A gives the statement and proof. We therefore estimate dimension from the delay vectors in controlled recurrent systems, and test their usefulness as a relative complexity measure in applied training.

2.3 Computing MG

Within a window of W scalar samples, we subtract the mean, divide by the standard deviation, and form the delay vectors. We then find their nearest neighbors, excluding vectors too close in time through a Theiler exclusion T (Theiler, 1986). This prevents adjacent samples from dominating the neighborhoods. Let k be the number of retained neighbors, r_{ij} the distance from vector i to its j th admissible neighbor, and n the number of valid query vectors. We compute

$$\hat{d}_{\text{MG}} = \frac{n(k-1) - 1}{\sum_{i=1}^n \sum_{j=1}^{k-1} \log(r_{ik}/r_{ij})}. \quad (3)$$

The distance ratios measure how quickly neighborhoods fill as their radius grows; faster growth corresponds to higher local dimension. Equation (3) pools this information across query vectors. The subtraction of one is a finite-sample correction under the local likelihood model, which assumes approximately uniform density at the neighborhood scale. Serial dependence, uneven excitation, and short records can still introduce bias. We retain these effects in the reported estimates rather than clipping values to E .

Scale invariance. For fixed delay and neighbor settings, the ideal estimator is invariant to $x_t \mapsto ax_t + b$, $a \neq 0$. Standardization removes the offset and amplitude scale, and the distance ratios remain unchanged. Thus MG can distinguish a change in temporal structure from a uniform change in amplitude. Appendix A proves this property and describes the effects of the small numerical noise added in the implementation. The invariance applies to a whole window; a scale change within that window can alter its temporal structure.

Windowing and interpretation. The delay span $S = (E-1)\tau$ determines the time interval represented by each vector. The window length W determines how much data enter an estimate and how precisely a transition can be located. Estimating dimension requires repeated visits within one regime. Monitoring requires a window short enough to distinguish the regimes before and after a change. We choose settings on calibration

records, hold them fixed during evaluation, and report detection delay. Appendix B lists the configurations.

Where measured, we report sensitivity to doubling the embedding dimension, $\rho_E = \hat{d}_{\text{MG}}(2E)/\hat{d}_{\text{MG}}(E)$, the frequency of numerical-floor corrections, and companion scalar statistics. For nearly constant records, we also check the amplitude before standardization. These checks expose sensitivity to the reconstruction and numerical noise. They do not establish recurrence or a local scaling law. Without those properties, we interpret MG through its agreement with the independent task-specific reference.

3 CONTROLLED EXPERIMENTS: RECOVERING COMPONENT STRUCTURE

We first test whether MG reflects the component structure that motivates the method. Analytic signals provide known dimensions; controlled learning problems test the effect of parameter updates; trained generators test whether a single neuron reveals the phase structure of a learned task. Section 4 then evaluates monitoring in applied learning problems.

3.1 Analytic oscillators and controlled learning

Known phases. In the analytic baseline, r diagonal entries of a matrix oscillate at rationally independent frequencies. The trajectory therefore has r independent phases. MG computed from a scalar observation tracks this count over $r = 1-8$, with mean absolute error 0.27. This establishes recovery against a known geometric target before introducing learning.

Learning on MLP features. A multilayer perceptron (MLP) with tanh activations is trained on handwritten digits (Pedregosa et al., 2011). We freeze its features and train a linear classifier in a fixed parameter subspace. Quasiperiodic changes in the loss weights of data groups provide a controlled source of recurrent motion. We select estimator settings on separate seeds and ranks, then evaluate ranks 1, 3, 5, 8 on new seeds.

Every usable observable orders the held-out ranks correctly with these fixed settings. Parameter norm and fixed-probe loss stop varying when the learning rate is zero, confirming that their variation requires learning. The mean absolute error against the independently measured covariance reference is 0.90 across rank-seed-observable combinations. This reference checks the excitation of parameter directions; it is not an exact active-dimension label. The result supports accurate ordering in a controlled learning task without retuning

the estimator for each rank. Appendix C gives errors for individual observables and distinguishes this task from earlier driven-target tests.

Endogenous oscillations in a regression problem. We next test whether MG detects fewer modes when all parameters remain trainable. We train all 32 weights of a dense linear regressor using full-batch heavy-ball updates. Repeated Hessian eigenvalues form r groups; momentum one produces r independent oscillatory modes with a fixed target. Selective damping reduces four modes to two without changing the data or trainable parameter count. MG of the training loss falls from 4.97 to 2.56. Multiplying the original loss by 0.01 leaves MG unchanged. The estimate therefore responds to the removal of modes in this experiment, while remaining insensitive to uniform rescaling. This is a controlled oscillatory optimizer: without damping, it oscillates around the solution rather than converging.

3.2 Learned generators distinguish phases from harmonics

The next test asks whether one neuron’s activity reveals the component structure learned by a recurrent network. A 1,000-unit rate network is trained by FORCE with recursive least squares (Sussillo and Abbott, 2009) to generate one to four sinusoidal outputs. Feeding the outputs back into the network makes the learned read-out affect its dynamics. At evaluation, the weights are fixed and the network runs without the target signal.

We train seven tasks on each of five seeds: one to four independent frequencies (T1–T4), two or four harmonics of one frequency (H2, H4), and two independent frequencies with their harmonics (M4). Their target phase counts are 1, 2, 3, 4, 1, 1, 2, respectively. All 35 networks meet the output-fidelity criterion. A Lyapunov analysis checks the learned dynamics independently. We choose MG settings on the target signals before analyzing the networks and use neuron zero as the primary observable.

The harmonic controls demonstrate the benefit of geometric analysis (Figure 1). H4 has four spectral lines but median MG is 1.15, close to the single-phase T1 task. For every seed, MG is lower for H2 than T2 and increases from H4 to M4 to T4. The latter three tasks have identical spectral peak counts and similar full-state covariance participation ratios: 5.27, 5.71, and 5.76. Their MG values are 1.15, 3.58, and 5.93. A single neuron’s temporal record thus distinguishes phase structures that these two summaries do not separate.

Across T1–T4, median MG is 1.15, 2.84, 5.90, and 5.93. It separates one- and two-phase tasks from the higher-dimensional tasks, but does not resolve T3 from T4.

The upward bias at higher phase counts limits absolute recovery at this record length. The reproducible result is discrimination of phase structure from one neuron, including the harmonic comparisons on all five seeds.

3.3 Learning a cycle from chaotic activity

In a separate FORCE experiment, a 256-unit network learns to produce a figure-eight trajectory from initially chaotic activity. The target has two outputs but one fundamental phase. We select five chaotic initializations using a pretraining Lyapunov criterion, without inspecting MG, and evaluate the network at saved checkpoints.

Single-neuron MG falls from 3.56–5.07 before training to 1.08–1.13 afterward in all five networks. The decline persists across three neurons and the tested window and lag settings. Independently, the full Lyapunov spectrum loses its clearly positive exponents. Four networks meet the stricter stable-cycle criterion; the fifth has weak contraction transverse to the cycle.

Both analyses identify simplification of the learned activity. MG does so from one neuron, while the spectrum uses the full state and tangent dynamics. Section 5 compares their computational costs. Here the measured trajectory is network activity at fixed weights, not the sequence of weight updates.

4 APPLIED EXPERIMENTS: MONITORING LEARNED SIMPLIFICATION

We evaluate three practical uses: detecting restrictions on parameter updates, tracking information loss in a representation, and measuring changes in learned behavior. Each experiment defines simplification through an independent reference. The test is whether MG detects that change from a scalar record, even when its absolute value has no established dimension interpretation.

4.1 CNN training: detecting restrictions on parameter updates

We train a 14,666-parameter convolutional neural network (CNN) on 10,000 CIFAR-10 images (Krizhevsky, 2009) for 10,000 SGD updates. At update 4,000, we reduce the learning rate, freeze layers, or prune weights. Controls continue unchanged, increase the batch size, or transform the observed log by rescaling or smoothing without altering training. MG receives only the norm of all parameters, which requires no extra network forward pass. Settings are $W = 1,000$, stride 500, $E = 20$, $\tau = 1$, and $k = 20$.

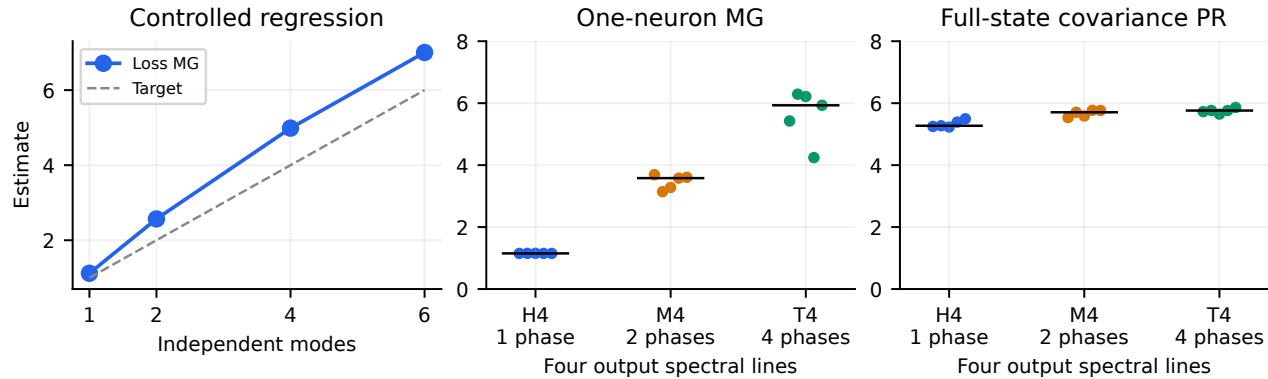


Figure 1: MG detects component structure from scalar records. Left: regression with a known number of modes and fixed estimator settings across ranks. Center: learned generators with four output spectral lines but one (H4), two (M4), or four (T4) independent target phases; each dot is a seed. MG separates these tasks. Right: their full-state covariance participation ratios are similar. The comparison shows sensitivity to phase structure, which differs from the spread of variance measured by participation ratio.

We calibrate on seeds 0–3 and test interventions of different strengths on new seeds 10–13. Independent measurements record the covariance participation ratio of updates, the fraction of parameters that move, and update magnitude. Freezing and pruning impose known restrictions on trainable or nonzero parameters. Learning-rate reduction changes the dynamics, but need not reduce invariant dimension; we assess its effect through the measured updates.

MG falls after strong restrictions on training. Freezing all but the classification head reduces it by a median 18.5%, while update participation ratio falls by 43.5%. Strong freezing and pruning produce median MG declines of approximately 12–19%; control medians range from near zero to increases of 13%. Across intervention strengths, MG ratios and update participation-ratio changes have Spearman correlation 0.59. They measure different aspects of the change: after 50% pruning, MG falls while update participation ratio rises.

MG adds information beyond simple scalar statistics. MG separates the graded interventions from unchanged-training and increased-batch controls with area under the ROC curve (AUC) 0.93. Trend crossings, lag-one autocorrelation, and detrended standard deviation achieve 0.48–0.57 on the same comparison. Comparing only the strongest intervention doses against all specified controls gives MG an AUC of 0.99.

As a further check, we apply MG to surrogate records generated by iterative amplitude-adjusted Fourier transforms (IAAFT). These preserve the marginal distribution and approximately the spectrum, but yield AUC 0.68. The stronger separation in the original records suggests that these properties alone do not explain the result. All comparisons use the same records

and a direction of change fixed before evaluation.

Detecting changes using only past observations.

For window estimates m_j , we compare the median over the latest M windows with the median over the preceding B windows:

$$D_j = \frac{\text{median}(m_{j-M+1}, \dots, m_j)}{\text{median}(m_{j-M-B+1}, \dots, m_{j-M})} - 1. \quad (4)$$

Calibration selects $M = 2$, $B = 3$, and an alarm threshold of $D_j < -0.0883$, a drop of 8.83%. Only completed windows enter the rule. We count the first threshold crossing as an alarm and score it as a detection if it occurs within 5,000 updates after the intervention. An earlier alarm is false. Competing statistics use the same calibration procedure.

MG detects 27/28 strong interventions with no alarms in 16 control records and median delay 1,000 updates (Table 1). Standard deviation detects all 28 but raises alarms in 10/16 controls; trend crossings detect 15/28 with 5/16 control alarms. MG therefore combines high detection with fewer false alarms on this benchmark. Weak interventions are less reliably detected. The records share four seeds, so these counts are not independent trials. We evaluate detection here; whether acting on an alarm improves training requires a separate test.

4.2 Transferring the detector to ResNet-18

We transfer the parameter-norm observable, MG settings, and alarm threshold to ResNet-18 (He et al., 2016), without recalibration. Training uses the full CIFAR-10 training set, either from scratch or from ImageNet-pretrained weights, with three new seeds in

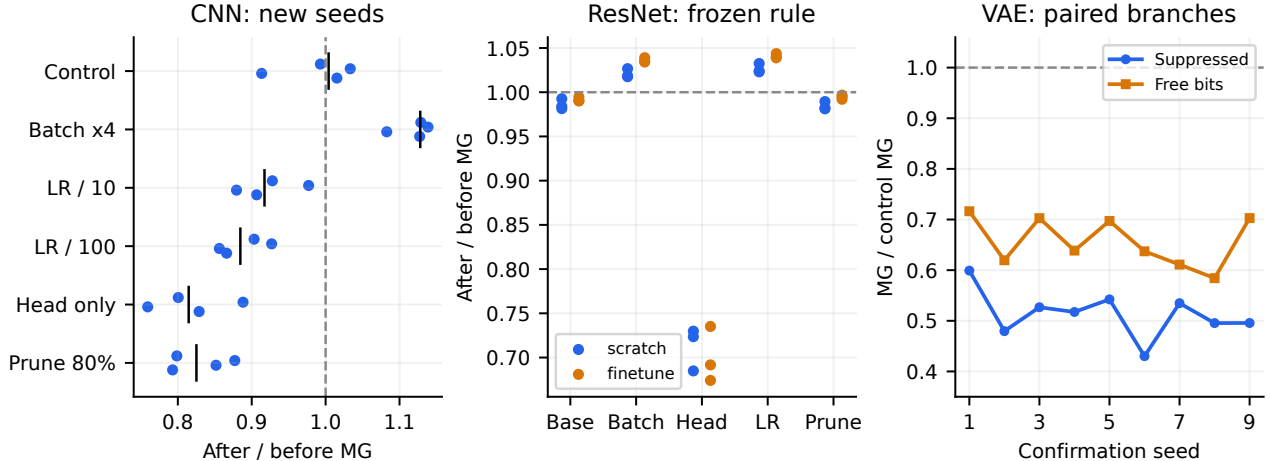


Figure 2: MG tracks changes across learning tasks. Left: CNN estimates after an intervention divided by those before it, on four held-out seeds. Bars show medians; the dashed line means no change. Center: the same comparison for ResNet-18 trained from scratch or pretrained weights, with all seeds shown. Right: late MG in the suppressed and free-bits VAE branches relative to their shared low-regularization control, across nine test seeds. Independent update and information measurements validate the changes.

Table 1: MG detects most strong CNN interventions with no control alarms. All methods are calibrated on the same earlier seeds and evaluated on four new seeds. “Alarms” counts control records with an alarm. Delay is measured only for detected events.

Statistic	Detected	Alarms	Delay
MG	27/28	0/16	1,000
Trend crossings	15/28	5/16	1,000
Lag-one correlation	0/28	0/16	—
Detrended std.	28/28	10/16	500

each setting. Freezing the backbone while continuing to train the head reduces MG by a median 27.6% and 30.8%, respectively. The detector identifies all six freezing events with no alarms in 24 control records.

This transfers a detector from about 15 thousand to 11 million parameters for a concrete training change. The global parameter norm is informative for backbone freezing, but the same detector misses the tested learning-rate reductions and pruning interventions in ResNet-18. Separately explored layer norms and random projections introduce false alarms. Thus transfer succeeds for the freezing task, while other interventions require further validation of the observable and detector.

4.3 Text VAE: tracking information loss and preservation

A text variational autoencoder (VAE) learns to reconstruct Penn Treebank sentences (Marcus et al., 1993)

through a 16-dimensional Gaussian latent code. Its encoder and decoder use gated recurrent units (Bowman et al., 2016); the model has 276,016 parameters and trains on 6,000 sentences. This task tests whether MG can reveal when the decoder stops using information in the code.

After 1,024 shared updates with Kullback–Leibler (KL) regularization weight $\beta = 0.01$, we split training into three branches: keep that weight, raise it to one, or use $\beta = 1$ with a free-bits objective. Free bits reduces pressure to suppress low-KL latent coordinates. All branches train until update 3,072.

MG uses the reconstruction negative log-likelihood (NLL) per token on 16 fixed validation sentences, with fixed latent sampling noise. It has no access to KL or the independent references. On a larger set of 256 validation sentences, the references estimate latent mutual information and the decoder’s response to shuffling codes between sentences. A joint decline means that less information is retained and used through the latent code.

The information-loss criterion is met on all nine test seeds. MG declines more strongly in the high-regularization branch than in its control on every seed: their median late MG ratio is 0.517, with a 95% seed-bootstrap interval [0.480, 0.542]. Free bits partially preserves both information and decoder dependence. MG is then higher than in the suppressed branch on all nine seeds, with median ratio 1.286 [1.179, 1.418]. The preservation control is informative because the two branches use the same external KL weight but retain

different amounts of information. MG distinguishes these outcomes from reconstruction logs alone.

This result validates MG as a signal of latent information loss and partial preservation. Standard deviation and spectral entropy also detect the loss on all nine seeds. In a separate cyclic-VAE test, using MG to schedule expensive reference checks finds 28/41 transitions at the primary budget, compared with 38/41 for periodic checks. Association with information loss therefore does not by itself imply better scheduling. The CNN benchmark provides the clearest detection advantage over simple scalar statistics.

4.4 Grokking and learned control

Grokking. In modular-arithmetic training, scalar parameter-norm MG falls near generalization in all four positive runs when its temporal resolution is matched to that of the trajectory analysis. Independently, saved trajectory sketches show a sharp, reversible collapse of detrended covariance participation ratio. Across settings applied to these same runs, 24/26 usable comparisons distinguish the positive runs from two controls. These comparisons are not independent repetitions. Roughness changes are stronger, a memorizing control also simplifies, and the full-batch setting does not reproduce the same scalar result. The evidence supports sensitivity to a training transition, without making an MG decline a unique indicator of generalization.

Learned locomotion. We also analyze behavior learned by proximal policy optimization (PPO) in Walker2d (Schulman et al., 2017; Todorov et al., 2012). At fixed policy checkpoints, we compute MG from a scalar joint or control signal. Two series of five fine-tuning seeds show lower control-log MG under moderate action smoothing. In the final comparison, median ratios to the control are 0.801 and 0.604 for two penalty strengths, with a decline on every seed. This extends the scalar analysis from training logs to learned behavior. Full-state repeatability and perturbation tests show that simpler control signals do not consistently imply a simpler full-body gait. Simple increment and return statistics remain competitive; Appendix E gives the comparisons and shared-initial-policy design.

5 COMPUTATIONAL SAVINGS AND CONDITIONS OF USE

Analysis from a compact record. The main computational advantage is the input: MG processes one scalar per step. Once that log is available, it needs no weights, activations, or Jacobian products. Its storage and nearest-neighbor calculations depend on W , E , and the temporal exclusion, not on the network’s pa-

Table 2: Time to analyze an available record. The FORCE reference is the full Lyapunov spectrum; the VAE reference combines latent-information and code-shuffling measurements. FORCE ranges cover before and after training. Compare methods within a row: tasks use different records and configurations. Scalar acquisition is excluded.

Setting	MG	Reference
CNN, 1,000-point window	9.5 ms	—
FORCE, 256 units	0.60–0.84 s	11.55–12.01 s
VAE, 512-point window	12.89 ms	209.32 ms

rameter count. Obtaining the log has a separate cost: a parameter norm requires an $O(P)$ reduction over P parameters, and a fixed-probe loss requires a forward pass. We therefore report acquisition and analysis separately.

For the CNN, the scalar record occupies about 39 KiB, compared with 559 MiB for the stored float32 parameter trajectory. Each 1,000-point window takes about 9.5 ms to process (Table 2). This makes repeated analysis inexpensive in the tested implementation. A streaming covariance calculation or trajectory sketch would reduce the reference’s storage requirement; the comparison is with the saved full trajectory.

Savings relative to full Lyapunov spectra. In the 256-unit recurrent network, both MG and the full spectrum detect the learned transition from chaos toward a cycle. MG takes 0.60–0.84 s, compared with 11.55–12.01 s for the spectrum: a 14–19-fold speed advantage. Including an additional MG calculation at doubled embedding dimension retains a 5–10-fold advantage. A separate 128-oscillator benchmark gives a 22–23-fold advantage over a dense full-spectrum calculation on a 1,536-point record.

These savings depend on the required information and window length. The largest Lyapunov exponent and elementary scalar statistics are faster than MG in the recurrent-network benchmark, but do not provide the full spectrum. MG does not recover individual exponents either. Longer scalar windows can also reverse the timing comparison; Appendix F reports these cases.

When logging costs dominate. For the VAE, an MG calculation is 16.2 times faster than the combined information and code-shuffling reference. Yet repeatedly evaluating the fixed probe dominates the total cost. At the primary cyclic-monitoring budget, the estimated total is 54.68 s for MG-triggered checks versus 2.72 s for periodic checks. Reusing an already available probe record reduces the MG total to 3.54 s. Thus a cheap estimator does not guarantee cheaper monitoring: its practical benefit depends on whether the informative

scalar is already logged or must be acquired specifically for analysis.

Where the method adds value. The experiments establish three complementary benefits. MG distinguishes phase structure that peak counts and covariance summaries miss; detects strong CNN interventions with fewer false alarms than standard deviation or trend crossings; and reduces analysis cost relative to full Lyapunov spectra in the specified benchmarks. The first benefit connects to the theory. The latter two make it useful for monitoring even where an exact component count is unavailable.

Choosing a useful observable. The observable must reflect the change of interest. Mini-batch noise can hide a change, while drift or numerical noise can dominate slow or nearly constant records. Transfer also depends on the event: in a separate ReLU-death test, the CNN-calibrated MG detector finds 12/21 eligible events with 12/31 control alarms, compared with 19/21 and 7/31 for trend crossings. In the tested edge-of-stability setting, MG does not track the count of near-threshold Hessian modes. These results define a practical validation step: check each new observable against an independent reference and a simple scalar baseline before relying on its alarms.

6 CONCLUSION

We developed and evaluated a method for monitoring simplification from scalar logs. Delay-embedding theory motivates its active-component interpretation, and controlled experiments test that interpretation against known phase structure. Applied experiments show that the same estimator detects restrictions on training, tracks latent information loss and preservation, and responds to changes in learned behavior. Its advantages are access to temporal geometry from one scalar, a favorable detection–false-alarm balance on the CNN benchmark, and measured savings relative to costly full-state analyses. Together, these results make MG a useful addition to training diagnostics when the observable is validated for the change being monitored.

AI USE STATEMENT

The authors formulated the central research questions, hypotheses, and scientific claims. Generative AI tools assisted with experimental design, implementation, debugging, analysis, figures, and reports, as well as drafting, translation, and editing. The authors checked all reported results, read the complete manuscript, and further revised the text. They take responsibility for the final text, claims, code, and experimental results.

REFERENCES

- Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7319–7328, 2021.
- Alessio Ansuini, Alessandro Laio, Jakob H. Macke, and Davide Zoccolan. Intrinsic dimension of data representations in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 10–21, 2016. doi: 10.18653/v1/K16-1002.
- Jeremy M Cohen, Simran Kaur, Yuanzhi Li, J Zico Kolter, and Ameet Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=jh-rTtvkGeM>.
- Elena Facco, Maria d’Errico, Alex Rodriguez, and Alessandro Laio. Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Scientific Reports*, 7(1):12140, 2017.
- Peiran Gao, Eric Trautmann, Byron M. Yu, Gopal Santhanam, Stephen Ryu, Krishna Shenoy, and Surya Ganguli. A theory of multineuronal dimensionality, dynamics and measurement. *bioRxiv* 214262, 2017. doi: 10.1101/214262.
- Guy Gur-Ari, Daniel A. Roberts, and Ethan Dyer. Gradient descent happens in a tiny subspace. *arXiv preprint arXiv:1812.04754*, 2018.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Elizaveta Levina and Peter J. Bickel. Maximum likelihood estimation of intrinsic dimension. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 17, 2004. URL <https://proceedings.neurips.cc/paper/2004/file/74934548253bcab8490ebd74afecebb5-Paper.pdf>.
- Chunyuan Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. In *International Conference on Learning Representations (ICLR)*, 2018. URL <https://openreview.net/forum?id=ryup8-WCW>.
- David J C MacKay and Zoubin Ghahramani. Comments on ‘maximum likelihood estimation of intrinsic dimension’ by e. levina and p. bickel, 2005. URL <http://www.inference.org.uk/mackay/dimension/>. Unpublished note.
- Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Computational Linguistics*, 19(2):313–330, 1993.
- Fabian Pedregosa et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Phil Pope, Chen Zhu, Ahmed Abdelkader, Micah Goldblum, and Tom Goldstein. The intrinsic dimension of images and its impact on learning. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=XJk19XzGq2J>.
- Tim Sauer, James A. Yorke, and Martin Casdagli. Embedology. *Journal of Statistical Physics*, 65(3-4): 579–616, 1991.
- Thomas Schreiber and Andreas Schmitz. Improved surrogate data for nonlinearity tests. *Physical Review Letters*, 77(4):635–638, 1996.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- Jaroslav Stark. Delay embeddings for forced systems. I. Deterministic forcing. *Journal of Nonlinear Science*, 9(3):255–332, 1999. doi: 10.1007/s003329900072.
- Jaroslav Stark, David S Broomhead, ME Davies, and J Huke. Delay embeddings for forced systems. II. Stochastic forcing. *Journal of Nonlinear Science*, 13(6):519–577, 2003. doi: 10.1007/s00332-003-0534-4.
- David Sussillo and L. F. Abbott. Generating coherent patterns of activity from chaotic neural networks. *Neuron*, 63(4):544–557, 2009. doi: 10.1016/j.neuron.2009.07.018.

Floris Takens. Detecting strange attractors in turbulence. In *Dynamical systems and turbulence, Warwick 1980*, pages 366–381. Springer, 1981. doi: 10.1007/BFb0091924.

James Theiler. Spurious dimension from correlation algorithms applied to limited time-series data. *Physical Review A*, 34(3):2427–2432, 1986.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.

PAPER CHECKLIST

1. For all models and algorithms presented, check if you include:
 - (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes] Sections 2–4 and the appendices.
 - (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [Yes] Scale invariance, window requirements, storage and empirical cost are discussed; no universal sample-complexity bound is claimed.
 - (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [No] The manuscript bundle contains figure/table reconstruction scripts and numerical evidence; a complete anonymized training-software release is separate.
2. For any theoretical claim, check if you include:
 - (a) Statements of the full set of assumptions of all theoretical results. [Yes] The propositions state their assumptions. Embedding results are cited as motivation and are not claimed to hold for every training record.
 - (b) Complete proofs of all theoretical results. [Yes] Appendix A proves the two elementary propositions stated here; established embedding theorems are cited.
 - (c) Clear explanations of any assumptions. [Yes] Sections 2 and A distinguish geometric interpretation from empirical monitoring.
3. For all figures and tables that present empirical results, check if you include:
 - (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [Yes] The source bundle includes saved numerical outputs and scripts to reproduce the plotted and tabulated results. It does not include all weights or raw corpora needed to repeat every training run.
 - (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [Yes] Experiment-specific descriptions and calibration splits are in the appendices and accompanying protocols.
 - (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes] All figures show individual records or seeds; ratios, medians, counts and the VAE bootstrap are defined explicitly.
- (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [No] CPU/thread settings are specified for the main timing comparisons, but a complete hardware inventory for every legacy run is not available in this manuscript.
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
 - (a) Citations of the creator If your work uses existing assets. [Yes] Dataset/model and methodological sources are cited in the text and asset notes.
 - (b) The license information of the assets, if applicable. [No] A consolidated license inventory is not included in this version.
 - (c) New assets either in the supplemental material or as a URL, if applicable. [Yes] Numerical evidence and manuscript-reconstruction scripts are provided with the source bundle.
 - (d) Information about consent from data providers/curators. [Not Applicable] No newly collected participant data are used.
 - (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Not Applicable] No new textual or image dataset is released; the bundled evidence is numerical.
5. If you used crowdsourcing or conducted research with human subjects, check if you include:
 - (a) The full text of instructions given to participants and screenshots. [Not Applicable]
 - (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Not Applicable]
 - (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Not Applicable]

A GEOMETRIC INTERPRETATION AND NUMERICAL IMPLEMENTATION

A.1 What follows from an embedding

The dimension interpretation has two steps. First, delay-embedding theorems give conditions under which a generic scalar observable can reconstruct the state of a deterministic system. Second, a local dimension estimator approximates the geometry of that reconstruction from finite data. The reconstruction theorem alone does not guarantee accurate estimation; the estimator alone does not establish an embedding.

Proposition 1 (Bi-Lipschitz invariance). *Let μ have local dimension d at s , and let F be bi-Lipschitz on its support, with constants $0 < c \leq C < \infty$. Then the pushforward $F_{\#}\mu$ has local dimension d at $F(s)$.*

Proof. For sufficiently small ϵ , the inverse image of $B(F(s), \epsilon)$ is bounded on the support between $B(s, \epsilon/C)$ and $B(s, \epsilon/c)$. Their probabilities bound the pushforward probability. The logarithms of c and C contribute vanishing constants after division by $\log \epsilon$, so both bounds have the same limiting exponent d . $\square$

This elementary property states the geometric assumption behind our interpretation. A smooth embedding of a compact smooth manifold provides the needed local regularity. For a fractal support, an injective map alone need not provide the required control of distances. The forced-system results of Stark (1999); Stark et al. (2003) impose additional conditions and include fibre-wise formulations. Their application to a stochastic optimizer would require checking those conditions; it does not follow from observing a scalar training log.

A.2 Affine invariance and its implementation boundary

Proposition 2 (Affine invariance of the ideal scalar estimate). *For a finite nonconstant record, fixed delay and neighbor settings, and no numerical dither, Eq. (3) is unchanged by replacing every observation with $ax_t + b$, for any $a \neq 0$, provided the estimate is defined.*

Proof. After centering and dividing by standard deviation, the transformed series is the original standardized series multiplied by $\text{sign}(a)$. All pairwise Euclidean distances of its delay vectors are identical. Temporal exclusions and admissible neighbors therefore remain unchanged, as do all distance ratios and their pooled estimate. $\square$

The implemented code adds Gaussian dither of scale 10^{-9} after standardization, floors distances at 10^{-8} and local log-ratio sums at 10^{-5} , and records the fractions affected by these floors. Positive affine changes preserve the standardized input in exact arithmetic and preserve a run with the same dither. A negative rescaling preserves the distribution under symmetric dither, but need not give exactly equal values with the same realized dither. Finite precision and almost-constant records require separate checks. The proposition concerns a whole-window affine transformation: a scale change within a window is a different time series.

A.3 Pooling and dependence

The numerator $n(k-1) - 1$ is the bias correction used by the existing implementation. Its Gamma-model motivation assumes idealized independent local statistics. Neighbor reuse and serial dependence mean that this correction is not a general finite-sample unbiasedness guarantee for the records in this paper. We use the same implementation throughout, preserve degeneracy flags, and do not clip large estimates to E . The Theiler exclusion removes temporally adjacent candidates but does not make the remaining observations independent.

B CONFIGURATIONS AND REPRODUCIBILITY

Tables and figures are generated from saved CSV/JSON experimental outputs. The accompanying source bundle records file hashes and maps numerical claims to their sources. We specify the observable, estimator settings, and calibration/test split for each task. All tasks use the same estimator; window length and lag are chosen for the relevant timescale.

Selecting a window. A window must leave enough admissible neighbors after delay reconstruction and temporal exclusion. Meeting the software minimum is necessary but does not guarantee an accurate estimate. For dimension estimation, the window should contain repeated visits within one regime. The lag must also expose the phase structure: increasing record length cannot compensate for delay vectors that cover only a nearly linear segment. For detection, longer windows stabilize estimates but blur transitions. We report detection delay to make this tradeoff explicit.

Units of replication. The CNN interventions share seeds and pre-intervention histories; rescaling and smoothing are transformations of control logs. Generator tasks reuse initialization seeds. The VAE branches share initial trajectories, data, and probes. The Walker continuations share a learned starting policy. We

Table 3: Primary configurations. An exclusion of “span” is $(E - 1)\tau$. The adaptive generator lag is one quarter of the autocorrelation time, capped so the delay span occupies at most one eighth of the window; its exclusion is capped at 320. Alternative settings are reported as sensitivity checks, not substituted for the primary results.

Experiment	Observable	W	E	τ	k	Theiler exclusion
Driven digits head	fixed state-based observers	8,000	20	4	20	at least span; cap 150
Known-mode regression	training loss / fixed probe	2,048; 8,192	16	1	10	100
Learned generator E9	activity of neuron zero	8,192	20	adaptive	20	autocorrelation; cap 320
CNN / ResNet-18	parameter norm	1,000	20	1	20	span
VAE	fixed-probe reconstruction NLL	512	20	1	20	39
Walker2d	joint / control scalar	2,048	20	8	20	312

present counts and paired contrasts at these levels; we do not treat overlapping windows or transformed logs as independent draws from a population. The VAE intervals are percentile bootstrap intervals over nine paired confirmation seeds, using 20,000 resamples and bootstrap seed 20260929. They measure initialization/batch variation conditional on the selected data and observable.

C CONTROLLED SYSTEMS AND MLP-BASED RECOVERY

C.1 Analytic and driven-learning validation

The controlled suite includes an analytic matrix with independent oscillatory entries and learning systems with time-dependent targets or forcing: online regression, logistic regression, a frozen nonlinear decoder, and an MLP in a parameter subspace. To distinguish input variation from learned motion, we set the learning rate to zero and check whether each observable still changes. Several early loss and gradient observables continue varying. Their dimension estimates can recover the forcing structure, but cannot be attributed solely to changing parameters. The digits-head experiment below addresses this ambiguity with state-based observables that become constant when learning stops.

Table 4: Controlled validation with each experiment’s reference. Mean absolute error (MAE) averages rank-seed-observable combinations. The driven-target MLP signal still varies with frozen parameters; its error describes the driven signal, not parameter motion alone.

Construction	Range	MAE
Analytic oscillating matrix	1–8	0.27
Driven-target MLP subspace	1–20	0.92
Digits head, covariance reference	1–8	0.90

In the digits experiment, we train and freeze an MLP feature extractor, then train a linear head in a fixed affine parameter subspace. The cross-entropy loss assigns weights to twelve fixed data groups. Quasiperi-

odic modulation of selected group weights drives the recurrent motion. Calibration uses seeds 90–92 and ranks 2,4,6; evaluation uses seeds 0–3 and held-out ranks 1,3,5,8. The selected settings are $E = 20$, $\tau = 4$, $k = 20$, and an 8,000-point window. This configuration lies on the boundary of the search grid and is held fixed for evaluation; the search does not establish a global optimum.

All usable state-based observables order the held-out ranks correctly. The mean error against measured covariance effective rank is 0.90 across individual records; aggregating by rank before computing the error gives 0.382. We report the former because these aggregations answer different questions. Errors by observable are 0.60 for fixed-probe loss, 0.41 for parameter norm, 0.34 for a fixed parameter projection, and 1.81 for gradient norm. Instantaneous mini-batch loss continues varying with frozen parameters, while quantized probe accuracy is degenerate; neither supports the parameter-dimension interpretation. The controlled phases define the intended structure, and the covariance reference checks which directions are measurably excited.

C.2 Dense regression with endogenous modes

The data matrix has 256 rows and 32 columns, full column rank, and Hessian $H = X^\top X/256$. Orthogonal rotations make the data and learned weights dense. Eigenvalues are grouped into r repeated values, with a single excited direction within each group. For heavy-ball momentum one and step size one, the error in group j obeys

$$q_{j,t+1} = 2 \cos(\omega_j) q_{j,t} - q_{j,t-1}, \quad \lambda_j = 2(1 - \cos \omega_j).$$

The selected irrational frequencies produce r independent phases. All 32 original coordinates move. Three seeds vary bases, teacher directions, and probes, but the normalized training-loss dynamics are identical up to rounding for a fixed spectrum. These are coordinate/observer checks rather than three independent loss processes.

Selective damping sets momentum to 0.98 in two of

Table 5: Known-mode regression: median scalar MG. Settings are fixed across r . A longer record does not uniformly reduce bias.

r	Loss, 2,048	Loss, 8,192	Probe, 8,192
1	0.90	1.12	1.01
2	2.39	2.57	2.55
4	3.71	4.99	4.11
6	5.88	7.00	4.98

four groups at step 8,192. The comparison uses disjoint 8,192-point windows before damping and after the transition, with the latter starting at 12,288. Loss-based MG changes from 4.97 to 2.56, and the median probe estimate from 4.15 to 2.63. With ordinary momentum 0.99 on all groups, learning converges and late losses reach rounding scale. Standardization can then produce spurious high estimates; the absolute pre-standardization scale must be checked.

C.3 Learned autonomous generators

The 1,000-unit generator evolves as

$$\begin{aligned} \mathbf{h}_{t+1} &= 0.9\mathbf{h}_t + 0.1(J \tanh \mathbf{h}_t + U\mathbf{z}_t), \\ \mathbf{z}_t &= A^\top \tanh \mathbf{h}_t. \end{aligned}$$

Here $\mathbf{h}_t$ is the network state, J the recurrent matrix with gain 1.2, U the output-feedback matrix, and A the readout trained by FORCE/RLS for 100,000 simulation steps. Frequencies are proportional to $1, \sqrt{2}, \sqrt{3}, \sqrt{5}$ for T1–T4; H2/H4 use integer harmonics, and M4 uses $1, 2, \sqrt{2}, 2\sqrt{2}$. All seven tasks use seeds 1–5. After training, we discard 5,000 burn-in steps and record 32,768 autonomous samples. The reported MG value is the median over four disjoint 8,192-point windows on neuron zero. Neurons one and two provide fixed robustness checks.

The Lyapunov reference uses twelve tangent vectors and a near-zero threshold of -0.02 chosen in advance. Its count agrees with the task phase count in 31/35 cases. H2 has weakly contracting directions near the threshold and is overcounted in four seeds. Thus even this reference has finite-time ambiguity. MG has finite-resolution error as well: one periodic H2 record gives 0.73, T2 has median 2.84, and T3/T4 overlap. The reliable comparison is the distinction between phase structures, especially in the harmonic controls; exact recovery across $d = 1$ –4 is not established.

For the separate 256-unit motion experiment, gain is 1.5 and FORCE learns a figure-eight target for 40,000 simulation steps. Each frozen checkpoint is evaluated after 2,000 burn-in steps over 8,192 recorded states. The chaotic-start cohort consists of the first five eligible seeds among sequential candidates 6–30, selected by

a positive pretraining Lyapunov criterion; thirteen candidates were examined, yielding seeds 7,9,15,16,18. No MG value enters selection. The unselected initial seeds 1–5 are retained separately because some start non-chaotic. This is conditional evidence for learning from chaos, not an estimate of how often random networks become chaotic or learn a stable cycle.

D APPLIED TRAINING PROTOCOLS AND FULL COMPARISONS

D.1 CNN and ResNet interventions

The small CNN has three convolutional layers and 80 ReLU channels in total, no normalization, and 14,666 parameters. It uses 10,000 CIFAR-10 images, SGD step 0.02, momentum 0.9, weight decay $5 \cdot 10^{-4}$, and batches of 64 sampled with replacement. The graded study runs 10,000 updates with an intervention at update 4,000. The before/after summaries use windows lying within the designated pre/post intervals, excluding the immediate mixed transition.

The learning-rate interventions divide the step size by 3, 10, or 100. Freezing leaves the later layers, only the head, or only its bias trainable. Pruning masks 50%, 80%, or 95% of weights. Some interventions rebuild SGD for the remaining trainable parameters, which also changes optimizer state. Their measured effect therefore includes that change and cannot be attributed to parameter count alone. Increased-batch controls multiply batch size by four. Rescaling and smoothing transform the base log without retraining.

Detector calibration searches $M \in \{2, 3, 4\}$ and $B \in \{3, 4, 6\}$. For each pair, its drop threshold is the smallest giving no alarms on calibration nulls, plus a margin of 0.02. The selected pair maximizes calibration event detection, with shorter delay as a tie breaker. Companion-statistic directions are chosen on calibration data as well. The rule is frozen for the new graded seeds and for ResNet. The selected MG threshold is 0.0882988401. The primary 27/28 count covers seven strong intervention types on four seeds; weak types are reported separately. The 16 controls comprise base, increased batch, rescaled base, and smoothed base on the same four seeds. Thus these counts describe a matched benchmark rather than independent Bernoulli trials.

ResNet-18 runs use the full CIFAR-10 training set, new seeds 20–22, and scratch/pretrained settings. SGD uses momentum 0.9, weight decay $5 \cdot 10^{-4}$, initial step 0.02, and batch 128 without augmentation; the pre-trained setting uses 64×64 images and ImageNet normalization. MG and the rule transfer unchanged. A

Table 6: CNN interventions at all tested strengths, on four held-out seeds. Changes are medians of paired record ratios. Update covariance and the fraction of moving parameters provide independent comparisons. Stronger pruning does not always produce a larger MG decline.

Intervention	MG before	MG after	MG change	Update PR change	Moving parameters
No intervention	4.31	4.32	+0.4%	+40.3%	100.00%
Batch $\times 4$	4.31	4.85	+12.8%	-7.3%	100.00%
Log $\times 10$	4.31	4.32	+0.4%	+40.3%	100.00%
Smoothed log	4.31	4.44	+3.3%	+40.3%	100.00%
Step / 3	4.31	4.35	+0.9%	-5.4%	100.00%
Step / 10	4.31	3.95	-8.3%	-21.6%	100.00%
Step / 100	4.31	3.85	-11.5%	-20.9%	100.00%
Freeze first two convolutions	4.31	4.27	-1.8%	-13.6%	65.31%
Train head only	4.31	3.51	-18.5%	-43.5%	2.25%
Train head bias only	4.31	3.58	-16.7%	-68.4%	0.07%
Prune 50%	4.31	3.80	-12.7%	+14.7%	50.30%
Prune 80%	4.31	3.59	-17.5%	-10.3%	20.48%
Prune 95%	4.31	3.79	-11.5%	-45.3%	5.55%

1,024-coordinate CountSketch of updates provides a tractable covariance reference. The head-only intervention succeeds in both settings; learning-rate and pruning detection does not. The six freezing records and 24 controls reuse three seeds and two initial-model settings.

D.2 ReLU-death stress test

A separate experiment uses six new seeds 30–35 and abrupt learning-rate shocks, with doses chosen on a pilot using activation death and accuracy, not MG. A channel is counted as dead if it is zero on all 500 diagnostic images. An event requires an increase of at least 0.05 in the fraction of dead channels. There are 42 training runs, augmented by rescaled and smoothed base logs. Twenty-three runs meet the death criterion; two catastrophic whole-network failures are excluded from the prescribed eligible-event score and remain reported separately. This gives 21 eligible events and 31 non-event records.

Table 7: Boundary case: the transferred detector on ReLU death. Rules are fixed before the confirmation results.

Statistic	Detected / 21	Alarms / 31
MG	12	12
Trend crossings	19	7
Loss stagnation	21	8
Detrended std.	21	25
Lag-one correlation	0	0

MG changes are nonmonotone in channel death: moderate loss of channels and catastrophic disruption can move the estimate in opposite directions. The CNN success therefore concerns the tested sustained training interventions and is not a generic failure detector.

D.3 Text VAE and preservation control

We select 6,000 training and 256 validation sentences with seed 20260929. Each has 8–24 words before the end-of-sentence token. A 2,000-token vocabulary is built from the training data. Shared embeddings have dimension 48, both GRUs have hidden size 64, and the diagonal Gaussian latent code has dimension 16. The code enters the decoder’s initial state and every decoding step. Training uses teacher forcing, Adam with learning rate 0.001, batch size 32, gradient-norm clipping at five, and no dropout. The loss sums reconstruction NLL within each sentence, adds β times the KL term, and averages over sentences. Since the encoder observes the sentence, this reconstruction score differs from marginal generative likelihood and ordinary language-model perplexity.

The branches share weights, Adam state, and batch/noise generators at update 1,024. Before every optimizer update, we evaluate 16 fixed validation sentences with fixed Gaussian noise (seed 829). Early MG is the median of five windows ending at updates 512–1,024; late MG is the median of nine windows ending at 2,048–3,072. Both use stride 128.

Independent references are measured every 128 steps. The empirical-mixture mutual information (MI) estimate uses four fixed draws from each of 256 posteriors and is bounded above by $\log 256$ nats. The code-response score is the symmetric KL divergence between decoder token distributions with matched and cyclically shuffled posteriors, using matched noise and teacher forcing. We record information loss when both references fall below half their control values, provided their early values were nontrivial.

The preservation branch uses free bits: at $\beta = 1$, it floors the batch-averaged KL of each latent coordinate

at 0.5 nats before summing. We select this intervention on pilot seed zero using only the independent references. An earlier pilot with five extra encoder updates fails to preserve information and is retained as a negative result. Preservation requires both late references to retain at least half their early values and exceed the suppressed branch by at least a factor of two. All nine test seeds pass. This is a third continuation of each shared initialization, so the comparisons remain paired.

Table 8: Text VAE: late medians across nine confirmation seeds. MI and code response are independent of the MG calculation.

Branch	MI	Code response	MG
Low regularization	5.545	2.087	8.129
Suppressed code	0.327	0.021	4.172
Free bits	3.934	0.599	5.491

The suppression result is consistent across all nine seeds at lag one for window lengths 256, 512, and 1,024. At lag four, seven of nine seeds agree. The 1,024-point configuration has only one early window. Using ordinary mini-batch loss instead of the fixed probe gives the expected direction on only four seeds. Standard deviation and spectral entropy of the fixed probe decline on all nine. These comparisons show that a stable probe is important for detecting the information change in this experiment.

D.4 Cyclic-VAE monitoring

Fresh seeds 100–109 use the same architecture/data. Seed 100 calibrates thresholds; seeds 101–109 confirm them. After 1,024 steps at $\beta = 0.01$, three 2,048-step cycles linearly raise β from zero to one in their first halves, hold it at one in their second halves, and reset. Total length is 7,168 updates. Training is independent of monitor requests. References every 64 steps define loss when both normalized quantities are at most 0.5, and recovery when both are at least 0.65, each on two consecutive checks. The event time is the second check. There are 41 events with a complete 512-step detection horizon and two late events scored separately.

The monitor requests a reference when the absolute log change of its feature from the previous request exceeds its pilot-selected threshold, respecting a 128-step refractory period and a budget of 6,12, or 24 extra checks. Every method also receives one initial check. For each budget, thresholds are selected from the same fourteen-value grid on the pilot. Decisions use no future values. Periodic checks use evenly spaced grid points with no favorable phase selection.

At budget 12, MG makes 9.44 requests on average. Matching a periodic schedule to the exact request count

Table 9: Cyclic VAE: detected events out of 41 at all tested budgets. Primary budget is 12.

Method	$B = 6$	$B = 12$	$B = 24$
MG	18	28	36
Standard deviation	13	20	38
Spectral entropy	17	28	35
Training KL	8	13	31
Schedule	18	18	26
Periodic	12	38	40

per seed yields 34 detections, compared with MG’s 28; this post-hoc comparison uses request counts but no event times. Against the main periodic comparator, MG’s mean per-seed recall is lower by 24.4 percentage points, with bootstrap interval $[-36.7, -10.0]$. At budget six, MG detects 18 events versus 12 for periodic checks, although a rule based on the known training schedule also detects 18. MG is therefore competitive at the smaller budget but does not improve monitoring overall in this test.

E ADDITIONAL LEARNING SETTINGS

MNIST. A 15,018-parameter tanh MLP is trained on a fixed balanced 2,040-image subset after downsampling to 14×14 . SGD uses momentum 0.9, batch 64, and no weight decay. At step 2,048 of 4,096, one paired branch reduces the learning rate from 0.1 to 0.01. A fixed 100-image loss probe is selected on the pilot. Detrended covariance rank declines more than the control on all five new seeds; MG does so on four of five, with median contrast only 4.8%. The larger-window sensitivity is more favorable but does not replace the primary result.

Grokking. The original modular-arithmetic studies include regularized mini-batch and unregularized full-batch settings. Random projections of parameter and logit trajectories support detrended covariance analysis. A sharp collapse and later recovery aligns with generalization in four positive regularized runs, with two controls; a memorizing zero-weight-decay run can show simplification elsewhere. The short-window parameter-norm analysis reports a coincident decline after matching temporal bandwidth. Its 24/26 usable cells vary analysis settings on the same runs. The signal is not a unique indicator of generalization, and scalar roughness is an alternative explanation in this setting. We retain this as a complementary example rather than the sole applied validation.

Walker2d. PPO policies control six actuators in Walker2d-v5. Fixed-policy evaluations use a 512-step warm-up and 4,096 recorded steps. In phase-

conditioned variants, the observation is augmented by sine/cosine phase and the reward is changed to encourage an intended periodic behavior. Subsequent comparisons use a common initial locomotion checkpoint with separate fine-tuning seeds, not independently trained starting skills. Scalar joint and action logs are analyzed alongside full-state return errors, inter-cycle variation, and responses to injected perturbations.

Moderate action-smoothing penalties give the most reproducible scalar result: two five-seed series show lower aggregate control-log MG. In the final five-seed comparison, penalty strengths 0.25 and 1 give median MG/control ratios 0.801 and 0.604, with a decline on every seed. Normalized increments also decline in four and five seeds and are cheaper. In the stricter full-gait experiment, only two of five pairs meet the joint improvement criterion, and only one pair has lower MG under every prescribed reconstruction. Thus a simpler command signal is not sufficient evidence of a simpler or more stable whole-body gait. Timing advantages below concern repeated perturbation probes, not these elementary scalar competitors.

F TIMING PROTOCOLS AND INFORMATION REQUIREMENTS

Timing separates simulation/training, recording an observable, analysis of an existing record, and extra diagnostic interventions. The comparisons are local implementation benchmarks with the listed configurations. They do not establish a universal complexity ordering among all algorithms for the same scientific question.

CNN. The saved graded result reports median MG processing of 9.49 ms per 1,000-point window. A float32 scalar record of 10,000 updates takes 39.06 KiB; the corresponding 14,666-dimensional trajectory takes 559.46 MiB. Parameter-norm acquisition is an $O(P)$ reduction, not zero-cost logging. CountSketch update references in ResNet use 1,024 stored coordinates per update and require computing the sketch. A streaming reference would change the storage comparison.

FORCE. The 256-unit endpoint benchmark replays fixed-checkpoint autonomous trajectories. Median MG times before/after training are 0.835/0.598 s; full-spectrum times are 12.006/11.551 s. With both $E = 20$ and $E = 40$ calculations, scalar analysis takes 2.233/1.213 s. Scalar recording costs 0.102/0.166 s, including rollout; this is separate from analysis. The largest exponent costs 0.198/0.153 s and elementary scalar baselines 0.0009/0.0015 s. The full spectrum supplies information that a single exponent or scalar

score does not, and MG does not recover the exponents themselves.

In the 1,000-unit E9 generator, analysis of four 8,192-point windows takes 8–23 s across learned-task medians. The twelve-vector Lyapunov calculation takes 41–66 s; full-state PCA is about 0.2 s. The advantage over PCA is its input requirement, not time. The difference from the CNN’s millisecond windows is material and follows from record/configuration and implementation costs.

Coupled oscillators. A separate benchmark evaluates dense phase networks of 32,64,96,128 oscillators at two coupling levels using a shared saved 1,536-point trajectory for each comparison. Scalar analysis computes MG and LB together; the full spectrum integrates all tangent directions with QR steps. Medians use five scalar and three spectrum repetitions. At size 128, scalar times are 0.061/0.049 s and full-spectrum times 1.309/1.124 s. Scalar storage is 128 times smaller. At size 32, the full spectrum is faster. An additional same-seed 64-unit comparison with 4,096-point scalar windows also favors the spectrum (roughly 1.0–1.4 s versus 1.6–2.2 s), so the short-window speedup is not universal. The same-seed coupling sweep shows a collective contraction of the spectrum and MG approaching one after synchronization; finite-time exponent thresholds are not exact dimension labels. This is a dynamical-system benchmark, not neural-network training.

VAE. The cyclic benchmark uses one warmed process, two PyTorch threads, one estimator thread, and forty interleaved repetitions. Median costs are 7.134 ms per fixed probe, 12.890 ms per MG window, and 209.323 ms per combined reference. Cost totals multiply component medians by the actual operation counts, including 7,168 probe acquisitions and 105 feature calculations. They are estimates of deployment cost, not separately timed complete monitoring runs. Other jobs may share the machine. At budget 12, acquisition alone costs 51.14 s, leading to the 54.68 s MG total. Cheap statistics using the same probe incur the same acquisition cost. Periodic checks require no such probe stream.

Walker2d. In two fixed-horizon comparisons, MG of an existing scalar is approximately 21–25 times cheaper than repeated perturbed simulations; including the doubled-embedding check reduces this to about 10–11 times. These probes answer questions about stability that MG alone cannot answer. On the final scalar benchmark, MG costs 369 ms per window, versus 0.103 ms for spectral entropy, 3.782 ms for a scalar return statistic, and 0.048 ms for normalized increments. The data-saving property is shared by all four scalar methods.

G HOW THE EXPERIMENTS SUPPORT THE METHOD

The controlled tests connect MG to known phase structure and identify its finite-resolution limits. The applied tests ask a separate question: does the same estimator detect a change confirmed by an independent measurement? Its value for that task depends on reliable detection, false alarms, and cost, rather than exact integer recovery. We report comparisons where MG adds information or saves computation, alongside cases where a simpler statistic is sufficient or better. The resulting evidence supports the specified observables and tasks; a new application requires the same kind of validation.